

▼ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.5 LTS
Release:      24.04
Codename:     noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Collecting OMSimulator
  Downloading omsimulator-3.0.0.post116.tar.gz (7.6 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (2.32.4)
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (6.1.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Building wheels for collected packages: OMSimulator
  Building wheel for OMSimulator (pyproject.toml) ... done
  Created wheel for OMSimulator: filename=omsimulator-3.0.0.post116-py3-none-any.whl size=1345325 sha256=afb8938c
  Stored in directory: /root/.cache/pip/wheels/eb/b0/ee/5194adbce75a2efc0262c5c0377b28ecbc648628be707b5a4
Successfully built OMSimulator
Installing collected packages: OMSimulator
Successfully installed OMSimulator-3.0.0.post116
```

```
!pip install DyMat
```

```
Collecting DyMat
  Downloading DyMat-0.7.tar.gz (24 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: DyMat
  Building wheel for DyMat (setup.py) ... done
  Created wheel for DyMat: filename=dymat-0.7-py3-none-any.whl size=19077 sha256=71d685eab360d5b9734a001cbdd8140
  Stored in directory: /root/.cache/pip/wheels/bd/8e/7a/d63cada988615a826eecd5353c397a21762db0633d8372d7e3
Successfully built DyMat
Installing collected packages: DyMat
Successfully installed DyMat-0.7
```

```
!pip install git+https://github.com/janpeter19/FMU_explore.git
```

```
Collecting git+https://github.com/janpeter19/FMU_explore.git
  Cloning https://github.com/janpeter19/FMU_explore.git to /tmp/pip-req-build-hesd4dl3
  Running command git clone --filter=blob:none --quiet https://github.com/janpeter19/FMU_explore.git /tmp/pip-req-
  Resolved https://github.com/janpeter19/FMU_explore.git to commit 1b0b3884884c3231c10dad33b379c258a6764d10
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: fmu_explore
  Building wheel for fmu_explore (pyproject.toml) ... done
  Created wheel for fmu_explore: filename=fmu_explore-1.2.5-py3-none-any.whl size=29258 sha256=3ef5f36f76bfc35164
  Stored in directory: /tmp/pip-ephem-wheel-cache-rt_wpkei/wheels/98/33/d9/8669158b0d845b084c49212b2330ad684e92d7
Successfully built fmu_explore
Installing collected packages: fmu_explore
Successfully installed fmu_explore-1.2.5
```

▼ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github. Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmпы_explore.py

```
%%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

```
Cloning into 'BPL_TEST2_Batch'...
```

```
%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

▼ BPL_TEST2_Batch - demo

```
import OMSimulator as oms
```

```
import scipy.io
```

```
import DyMat
```

```
import matplotlib.pyplot as plt
```

▼ Basic simulation

```
fmu = oms.FMU('BPL_TEST2_Batch_linux_om_me.fmu')
```

```
fmu.modelName
```

```
'BPL.Examples_TEST2.Batch'
```

```
for s in fmu.states:
    print(s.name)
```

```
bioreactor.V
bioreactor.m[1]
bioreactor.m[2]
```

```
fmu.instantiate()
```

```
fmu.setResultFile('result.mat')
```

```
fmu.setValue('bioreactor.V_start', 1.0)
fmu.setValue('bioreactor.m_start[1]', 1.0)
fmu.setValue('bioreactor.m_start[2]', 10.0)
```

```
fmu.initialize()
```

```
fmu.simulate()
```

```
mat_data = scipy.io.loadmat('result.mat')
```

```
mat_data.keys()
```

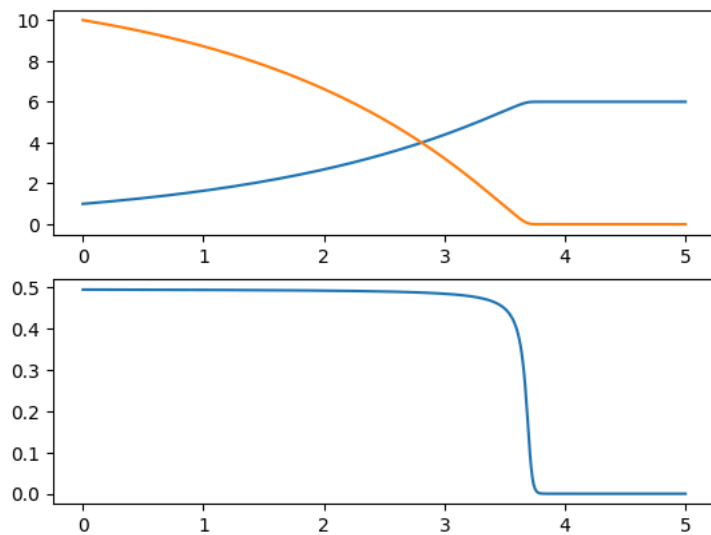
```
dict_keys(['Aclass', 'name', 'description', 'dataInfo', 'data_1', 'data_2'])
```

```
t = mat_data['data_2'][0];
```

```
plt.figure()

ax1 = plt.subplot(2,1,1)
ax2 = plt.subplot(2,1,2)

ax1.plot(t, mat_data['data_2'][2])
ax1.plot(t, mat_data['data_2'][3])
ax2.plot(t, mat_data['data_2'][12])
plt.show()
```



Simulation looks what I expected!

How to address variables using names

```
res = DyMat.DyMatFile('result.mat')
```

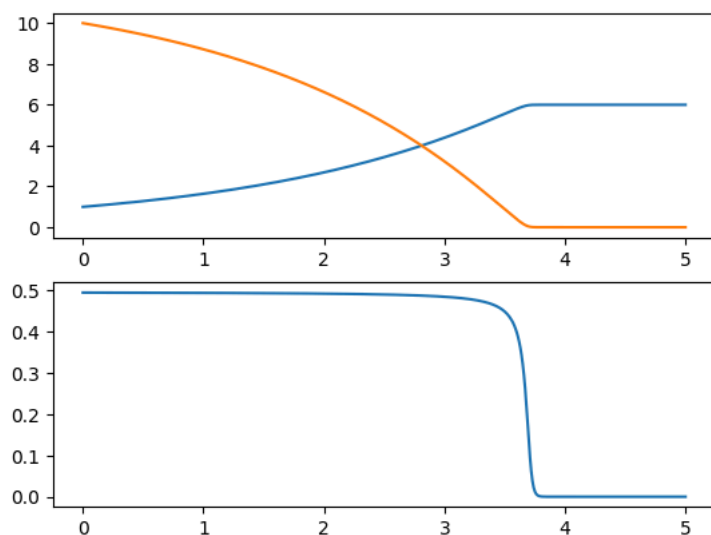
```
print(res.names())
```

```
dict_keys(['Batch.bioreactor.V', 'Batch.bioreactor.m[1]', 'Batch.bioreactor.m[2]', 'Batch.der(bioreactor.V)', 'Ba
```

```
plt.figure()

ax1 = plt.subplot(2,1,1)
ax2 = plt.subplot(2,1,2)

ax1.plot(t, res['Batch.bioreactor.m[1]'])
ax1.plot(t, res['Batch.bioreactor.m[2]'])
ax2.plot(t, res['Batch.bioreactor.culture.mu'])
plt.show()
```



How to address "time" in the res-file?

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Bring in FMU_explore

```
# Import the application model and context variables
from BPL_TEST2_Batch_oms import*
```

Linux – run FMU pre-compiled OpenModelica

```
model = None
opts_std = None
```

```
# Import the general FMU_explore functions ana adapt them to the application
from fmu_explore_oms import AdaptWith
```

```
Application = AdaptWith(parValue, parLocation, parCheck, \
                          model, fmu_model, fmu_process_diagram, \
                          MSL_usage, MSL_version, BPL_version, \
                          opts_std, simulationTime, timeDiscreteStates, \
                          diagrams, ax, lines)
```

```
par = Application.par
init = Application.init
```

```
process_diagram = Application.process_diagram
```

```
describe_MSL = Application.describe_MSL
FMU_explore_info = Application.FMU_explore_info
system_info = Application.system_info
SDG = Application.SDG
```

parValue

```
{'V_start': 1.0,
 'VX_start': 1.0,
 'VS_start': 10.0,
 'Y': 0.5,
 'qSmax': 1.0,
 'Ks': 0.1}
```

parLocation

```
{'V_start': 'bioreactor.V_start',
 'VX_start': 'bioreactor.m_start[1]',
 'VS_start': 'bioreactor.m_start[2]',
 'Y': 'bioreactor.culture.Y',
 'qSmax': 'bioreactor.culture.qSmax',
 'Ks': 'bioreactor.culture.Ks',
 'mu': 'bioreactor.culture.mu'}
```

parCheck

```
["parValue['Y'] > 0",
 "parValue['qSmax'] > 0",
 "parValue['Ks'] > 0",
 "parValue['V_start'] > 0",
 "parValue['VX_start'] >= 0",
 "parValue['VS_start'] >= 0"]
```

par(Y=0.4)

parValue

```
{'V_start': 1.0,
 'VX_start': 1.0,
 'VS_start': 10.0,
 'Y': 0.4,
 'qSmax': 1.0,
 'Ks': 0.1}
```

Start coding or [generate](#) with AI.

